

Neo4j 知识图谱和 Graph RAG 已付费

原创 神说要有光 神光的幸福生活 2026年5月9日 15:10 山东

我们学了 Milvus 可以做向量检索，检索出语义相近的文档，学了 Elasticsearch 可以做 bm25 的全文检索，检索出包含对应关键词的文档，但它们都有个问题：

无法捕捉数据之间的关联关系，只能实现“单点式”检索，难以应对需要挖掘数据内在逻辑、关联链路的场景。

比如语义检索，我们检索“奶茶推荐”，它能返回语义相似的奶茶相关文档，却无法告诉我们“奶茶”与“珍珠”“芋圆”“果糖”这些配料的关联，也无法串联起“奶茶品类→配料选择→热量高低→适合人群”的逻辑链路。

比如关键词检索，检索“珍珠奶茶”，它能返回所有包含“珍珠奶茶”的内容，却分不清“珍珠奶茶”与“台式奶茶”“港式奶茶”的品类关系，也无法梳理出“珍珠奶茶→配料珍珠→产地→制作工艺”的关联脉络。

Milvus 和 Elasticsearch 更像是“精准找货”的工具——一个按“语义相似”找，一个按“关键词”找，但当我们需搞清楚“货与货之间的关系”“货的来龙去脉”，比如在 Agent 开发的 RAG 场景中，需要基于文档关联推导答案、梳理知识体系，或者在日常消费检索中，需要串联品类、配料、使用场景之间的逻辑时，这两个工具就显得力不从心了。

而这，正是我们接下来要学习的 Neo4j 的核心优势——Neo4j 作为一款图数据库，它不关注“单个数据本身”，而是专注于存储和挖掘数据之间的关联关系，能把分散的文档、实体、关键词，像织网一样串联起来，形成清晰的知识图谱，完美解决 Milvus 和 Elasticsearch 无法捕捉关联关系的痛点。

比如我们把各类奶茶、配料、制作工艺、适合人群等信息都存入 Neo4j，就能轻松实现“检索珍珠奶茶→关联到配料珍珠→串联到制作工艺→延伸到适合的消费场景”的链路检索。

这也是我们做 RAG 优化、搭建完善知识库项目的关键一步。

这种基于图数据库实现的关联检索，就是 GraphRAG 的核心

GraphRAG 是“知识图谱+RAG”的结合体，它并非推翻我们之前学的传统 RAG，而是对 RAG 检索模块的升维优化，专门解决传统 RAG（依赖 Milvus 向量检索、ElasticSearch 全文检索）无法进行多跳推理、难以捕捉实体关联的痛点。

GraphRAG 会先将非结构化数据中的实体、关系提取出来，用 Neo4j 这样的图数据库构建知识图谱，再结合向量检索的语义优势，实现“图谱关联+语义匹配”的双重检索，既能找到

语义相近的内容，又能顺着知识图谱的关联链路，完成跨文档、多步骤的复杂推理，让 RAG 生成的答案更精准、更具可解释性。

传统 RAG 拿到的都是碎片化信息，没有结构、没有关联，像一个个信息孤岛。

而想要真正实现能推理、能关联、能解释的下一代 RAG，我们就必须用上知识图谱 + GraphRAG。

接下来，我们就正式进入 Neo4j 图数据库的学习

创建项目：

```
mkdir neo4j-graphrag  
cd neo4j-graphrag  
npm init -y
```

我们先创建下 docker-compose.yml, 跑一下 neo4j

```
services:
  neo4j:
    image: neo4j:latest
    container_name: neo4j-container
    ports:
      - "7474:7474"    # Web 管理界面
      - "7687:7687"    # Bolt 协议 (代码连接)
    environment:
      - NEO4J_AUTH=neo4j/12345678 # 账号: neo4j 密码: 12345678
      - NEO4J_PLUGINS=["apoc"]    # 安装必备插件
      - NEO4J_dbms_security_procedures_unrestricted=apoc.*
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/neo4j/data:/data
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/neo4j/logs:/logs
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/neo4j/conf:/conf
    restart: unless-stopped
```

跑一下:

```
docker compose up -d
```

neo4j 有自己的一套 cypher 的语句

创建 cypher.md

```
# Neo4j Cypher 语句实战：奶茶知识图谱

## 一、创建实体（节点）

### 1. 创建奶茶品类

CREATE (product:Product {name: "珍珠奶茶"})
CREATE (type1:Type {name: "台式奶茶"})
CREATE (type2:Type {name: "港式奶茶"})

### 2. 创建配料

CREATE (ing1:Ingredient {name: "珍珠"})
CREATE (ing2:Ingredient {name: "芋圆"})
CREATE (ing3:Ingredient {name: "果糖"})
CREATE (ing4:Ingredient {name: "红茶"})
CREATE (ing5:Ingredient {name: "牛奶"})

### 3. 创建制作工艺 & 适用人群

CREATE (method1:Method {name: "煮制"})
CREATE (method2:Method {name: "冲泡"})

CREATE (people1:People {name: "年轻人"})
CREATE (people2:People {name: "学生"})
CREATE (people3:People {name: "甜食爱好者"})

## 二、创建关系（知识图谱核心）

// 珍珠奶茶 属于 台式奶茶
MATCH (p:Product {name: "珍珠奶茶"}), (t:Type {name: "台式奶茶"})
CREATE (p)-[:属于]->(t)

// 珍珠奶茶 包含 配料
MATCH (p:Product {name: "珍珠奶茶"}), (i:Ingredient {name: "珍珠"})
CREATE (p)-[:包含]->(i)

MATCH (p:Product {name: "珍珠奶茶"}), (i:Ingredient {name: "果糖"})
CREATE (p)-[:包含]->(i)

MATCH (p:Product {name: "珍珠奶茶"}), (i:Ingredient {name: "红茶"})
CREATE (p)-[:包含]->(i)

MATCH (p:Product {name: "珍珠奶茶"}), (i:Ingredient {name: "牛奶"})
CREATE (p)-[:包含]->(i)

// 配料 使用 制作工艺
MATCH (i:Ingredient {name: "珍珠"}), (m:Method {name: "煮制"})
CREATE (i)-[:使用]->(m)
```

```
// 珍珠奶茶 适合 人群
MATCH (p:Product {name: "珍珠奶茶"}), (peo:People {name: "年轻人"})
CREATE (p)-[:适合]->(peo)

MATCH (p:Product {name: "珍珠奶茶"}), (peo:People {name: "学生"})
CREATE (p)-[:适合]->(peo)

MATCH (p:Product {name: "珍珠奶茶"}), (peo:People {name: "甜食爱好者"})
CREATE (p)-[:适合]->(peo)

## 三、查询验证

### 1. 查询全部节点与关系
MATCH (n)-[r]->(m)
RETURN n, r, m

### 2. 多跳关联查询 (GraphRAG 能力)
// 查询: 珍珠奶茶 → 配料 → 制作工艺
MATCH (p:Product {name: "珍珠奶茶"})-[:包含]->(i)-[:使用]->(m)
RETURN p.name, i.name, m.name

// 查询: 珍珠奶茶适合哪些人
MATCH (p:Product {name: "珍珠奶茶"})-[:适合]->(people)
RETURN p.name, people.name
```

跑一下:

新增和查询跑通，再试一下更新和删除：

创建 cypher2.md

1. 更新：给珍珠奶茶加热量属性

```
MATCH (p:Product {name:"珍珠奶茶"})
```

```
SET p.calorie = "中高热量", p.taste = "甜香"
```

2. 更新：修改珍珠工艺属性

```
MATCH (i:Ingredient {name:"珍珠"})
```

```
SET i.origin = "台湾", i.hard = "Q 弹"
```

3. 只删除某一条关系

```
// 删除 珍珠奶茶 适合 学生 这条关系
```

```
MATCH (p:Product {name:"珍珠奶茶"})-[r: 适合]->(s:People {name:"学生"})
```

```
DELETE r
```

4. 删除单个节点（无关联才可删）

```
MATCH (t:Type {name:"港式奶茶"})
```

```
DELETE t
```

5. 删除节点 + 连带所有关系

```
MATCH (i:Ingredient {name:"芋圆"})-[r]-()
```

```
DELETE r, i
```

6. 清空所有节点和关系（本地测试用）

```
MATCH (n)
```

```
DELETE n
```

会用在图形界面增删改查节点、关系之后，我们再用代码来试一下：

安装依赖：

```
pnpm install neo4j-driver
```

创建 src/neo4j-test.mjs

```
import neo4j from 'neo4j-driver'

// 连接信息 (和你的 docker-compose 完全一致)
const driver = neo4j.driver(
  'bolt://localhost:7687',
  neo4j.auth.basic('neo4j', '12345678')
)

// 获取会话
const session = driver.session()

// 1. 执行创建节点 (示例)
asyncfunction createData() {
  const result = await session.run(`
    CREATE (p:Product {name: "珍珠奶茶"})
    CREATE (i:Ingredient {name: "珍珠"})
  `)
  console.log('创建成功')
}

// 2. 执行创建关系 (示例)
asyncfunction createRelation() {
  await session.run(`
    MATCH (p:Product {name: "珍珠奶茶"}), (i:Ingredient {name: "珍珠"})
    CREATE (p)-[:包含]->(i)
  `)
  console.log('关系创建成功')
}
```

```

// 3. 查询数据
asyncfunction queryData() {
  const result = await session.run(`
    MATCH (p:Product {name: "珍珠奶茶"})-[r]->(i)
    RETURN p, r, i
  `)

  result.records.forEach(record => {
    console.log('奶茶:', record.get('p').properties.name)
    console.log('关系:', record.get('r').type)
    console.log('配料:', record.get('i').properties.name)
    console.log('-----')
  })
}

// 4. 更新属性
asyncfunction updateData() {
  await session.run(`
    MATCH (p:Product {name: "珍珠奶茶"})
    SET p.price = 15, p.calorie = "中高"
  `)
  console.log('更新成功')
}

// 5. 删除关系
asyncfunction deleteRelation() {
  await session.run(`
    MATCH (p:Product {name: "珍珠奶茶"})-[r:包含]->(i:Ingredient {name: "珍珠"})
    DELETE r
  `)
  console.log('删除关系成功')
}

// 6. 删除节点
asyncfunction deleteNode() {
  await session.run(`
    MATCH (p:Product {name: "珍珠奶茶"})
    DELETE p
  `)
  console.log('删除节点成功')
}

// 执行 (你想运行哪个就打开哪个)
// createData()
// createRelation()
queryData()

```

```
// updateData()  
// deleteRelation()  
// deleteNode()
```

驱动包连上之后，具体的 cypher 语句和之前一样

神光的编程秘籍

接下来，我们就可以基于刚学的 Neo4j 知识图谱来实现 Graph RAG 了

安装依赖：

```
pnpm install @langchain/community @langchain/openai @langchain/core @langchain/langgraph
```

创建 .env

```
OPENAI_API_KEY=sk-xx  
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1  
RERANK_URL=https://dashscope.aliyuncs.com/api/v1/services/rerank/text-rerank/text-rerank  
MODEL_NAME=qwen-plus
```

然后创建 src/graphrag.mjs

```
import 'dotenv/config'  
import { Neo4jGraph } from '@langchain/community/graphs/neo4j_graph'  
import { ChatOpenAI } from '@langchain/openai'  
import { StateGraph, END, START } from '@langchain/langgraph'
```

```

import { HumanMessage } from '@langchain/core/messages'

// -----
// 连接 Neo4j 知识图谱
// -----

const graph = new Neo4jGraph({
  url: 'bolt://localhost:7687',
  username: 'neo4j',
  password: '12345678',
})

// -----
// 大模型
// -----

const llm = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  temperature: 0,
  configuration: { baseUrl: process.env.OPENAI_BASE_URL }
})

// -----
// 定义状态
// -----

const state = {
  messages: {
    value: (left, right) =>
      left.concat(Array.isArray(right) ? right : [right]),
    default: () => [],
  },
  query: null,
  cypher: null,
  context: null,
  answer: null,
}

// -----
// 步骤1: 解析问题
// -----

asyncfunction parseQuestion(state) {
  const lastMessage = state.messages[state.messages.length - 1]
  return { query: lastMessage.content }
}

// -----
// 步骤2: 生成 Cypher
// -----

asyncfunction generateCypher(state) {

```

```
const prompt = `
你是一个专业的 Neo4j Cypher 生成器。

严格按照下面的结构生成正确语句，只返回纯 Cypher 代码，不要任何解释、不要标点、不要 markdown。
```

节点：

- Product：奶茶产品
- Ingredient：配料
- Type：奶茶类型
- Method：制作工艺
- People：适合人群

关系方向（必须严格遵守）：

- (Product)-[:属于]->(Type)
- (Product)-[:包含]->(Ingredient)
- (Product)-[:适合]->(People)
- (Ingredient)-[:使用]->(Method)

规则：

1. 关系方向绝对不能反
2. 多跳查询请使用多个 MATCH，不要连错路径
3. 只返回最终可运行的 Cypher 语句

用户问题：\${state.query}

```
`
const res = await llm.invoke([new HumanMessage(prompt)])
return { cypher: res.content }
}
```

```
// -----
```

```
// 步骤3: 执行图查询
```

```
// -----
```

```
asyncfunction executeGraphQuery(state) {
  try {
    const res = await graph.query(state.cypher)
    return { context: JSON.stringify(res) }
  } catch (e) {
    return { context: '未查询到相关知识' }
  }
}
```

```
// -----
```

```
// 步骤4: 生成答案
```

```
// -----
```

```
asyncfunction generateAnswer(state) {
  const prompt = `
```

你是奶茶专家，根据下方「检索结果」回答用户问题；检索结果为空或不足时简要说明无法从图谱得到答案，不要编回答要求：

- 直接列出事实，不要推断图谱里未出现的配料（如水、冰、添加剂等）。

检索结果: `${state.context}`

用户问题: `${state.query}`

```
const res = await llm.invoke([new HumanMessage(prompt)])
return { answer: res.content }
}

// -----
// 构建 LangGraph 工作流
// -----
const workflow = new StateGraph({ channels: state })
  .addNode('parse', parseQuestion)
  .addNode('generateCypher', generateCypher)
  .addNode('executeGraph', executeGraphQuery)
  .addNode('generateAnswer', generateAnswer)
  .addEdge(START, 'parse')
  .addEdge('parse', 'generateCypher')
  .addEdge('generateCypher', 'executeGraph')
  .addEdge('executeGraph', 'generateAnswer')
  .addEdge('generateAnswer', END)

const app = workflow.compile()

asyncfunction printWorkflowMermaid() {
const drawable = await app.getGraphAsync()
const mermaid = drawable.drawMermaid({ withStyles: true })
console.log('--- LangGraph 工作流 (Mermaid) ---')
console.log(mermaid)
console.log('-----')
}

// -----
// 运行 GraphRAG
// -----
asyncfunction runGraphRAG(question) {
const res = await app.invoke({
  messages: [new HumanMessage(question)],
})

console.log('=====')
console.log('用户问题: ', question)
console.log('生成 Cypher: ', res.cypher)
console.log('检索结果: ', res.context)
console.log('最终回答: ', res.answer)
console.log('=====')
```

```
}

// =====
// 测试
// =====

;(async () => {
  await printWorkflowMermaid()
  await Promise.all([
    runGraphRAG('我们这款珍珠奶茶有哪些配料? '),
    runGraphRAG('台式奶茶的饮品都有哪些配料? '),
    runGraphRAG('珍珠奶茶适合哪些人群饮用? '),
  ])
}()).catch(console.error)
```

跑一下：

我们问台式奶茶的产品有哪些配料

首先查了台式奶茶属于的类型，是珍珠奶茶

然后查了珍珠奶茶包含的配料

这就是知识图谱检索

这样，基于 Neo4j 知识图谱的 Graph RAG 就跑通了。

向量检索是语义检索、es 检索是分词后从倒排索引表通过关键词检索，知识图谱检索则是根据推理关系用 cypher 语句检索

都有各自的场景，都不可或缺

对比下这三种：

一、Milvus 向量语义检索

适合场景

- 用户提问没有明确关键词，是自然语言大白话；
- 需要语义相似、意思相近匹配，不是字面一样；
- 模糊查询、泛化查询、推荐类场景；
- 非结构化文档：笔记、手册、文章、FAQ 模糊问答。

不懂关键词、只看意思相近，交给 Milvus。

二、ElasticSearch BM25 关键词检索

适合场景

- 用户有明确专有名词、专业术语、编号、文件名；

- 需要精准分词、字面命中、高亮匹配；
- 官方文档、规章条款、接口文档、目录检索；
- 过滤、排序、时间筛选、字段精准匹配。

要精准匹配关键词、专业名词、固定术语，交给 ES。

三、Neo4j 知识图谱检索 (GraphRAG) 适合场景

- 需要实体关联、关系查询、多跳推理；
- 要查「A 和 B 什么关系、A 包含哪些、A 属于哪类」；
- 层级结构、分类体系、上下游、从属、配料、品类等链路查询；
- 传统检索给的是零散文本，需要逻辑推理、脉络梳理的场景。

要查关系、层级、脉络、多跳推理，交给知识图谱。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们明确了 Milvus 向量检索、ElasticSearch 的 BM25 全文检索的局限，只能实现单点检索，不能做多跳推理，梳理概念间的关联关系

所以我们要用 Neo4j 这种图数据库的知识图谱来存储节点、关系。

我们学了用 cypher 语句做阶段、关系的增删改查。

然后在代码里连接远程执行 cypher 语句

之后结合进 RAG 的流程，用 langgraph 来实现了 GraphRAG

其实就是把检索过程换成了生成 cypher 语句、检索、生成回答。

但是这个检索是知识图谱的检索，是有推理关系在的。

知识图谱的检索、ES 的关键词检索、Milvus 的语义检索都有各自适合的场景

三者短板刚好互补：

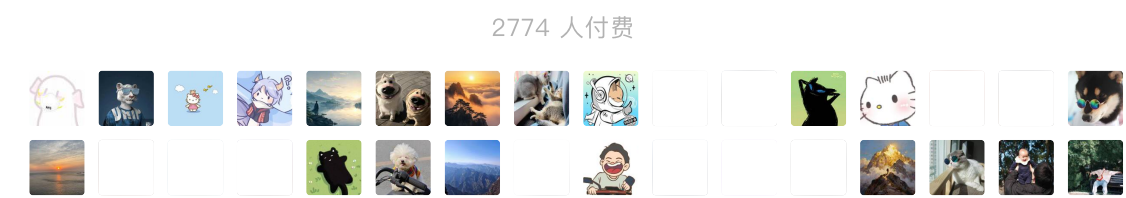
Milvus：擅长语义模糊匹配，但没有结构、不懂关系

ES：擅长关键词精准命中、分词倒排、过滤筛选，但也只是文本孤岛

Neo4j：擅长实体关联、多跳推理、层级脉络，但不擅长模糊语义、全文海量文档检索

组合之后：互相兜底、互相增强。

后面我们知识库项目会结合来用。



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇
混合检索 RAG：多路召回 + 重排模型

下一篇
LangSmith 全链路观测：从 Agent 调试到 RAG 量化评估